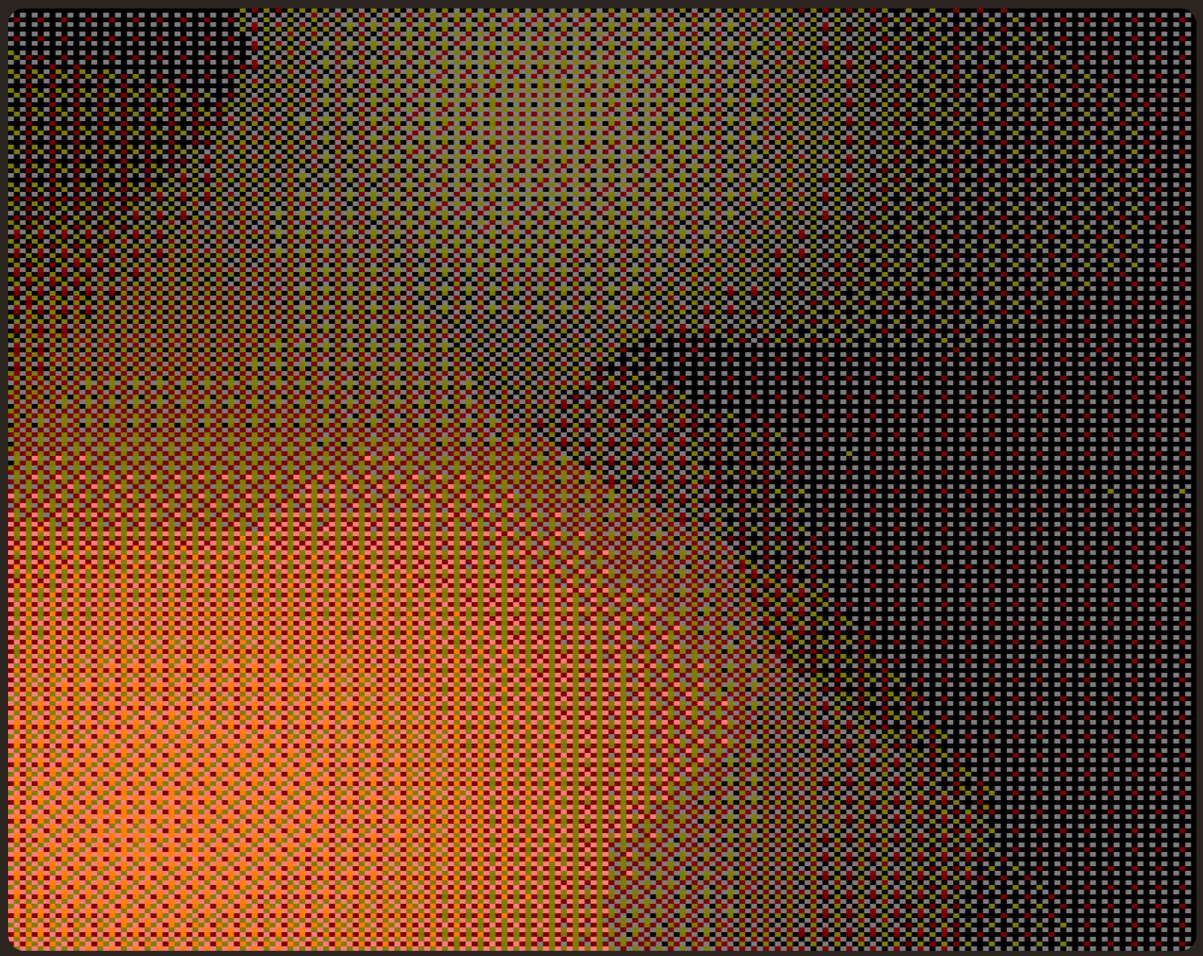
ZKSECURITY

E-ID ZKP Security Review

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE (EPFL)
CENTER FOR DIGITAL TRUST (C4DT)

SEPTEMBER 4TH, 2026 • TECHNICAL REPORT



§01

Introduction

On August 24th, 2026, zkSecurity started a security review of the zero-knowledge proof stack developed by EPFL and SICPA as part of the Innosuisse project Secure and Privacy-Preserving Credentials for E-ID. The project is a collaboration between EPFL's Center for Digital Trust (C4DT) and SICPA. The review lasted two weeks with one senior consultant and focused on the soundness, completeness, and privacy of the circuits for credential presentation and their custom proving backend.

The reviewed stack uses Noir circuits to prove properties of Swiss E-ID SD-JWT credentials, including issuer authentication, holder binding, age verification, and non-revocation, without disclosing the complete credential. The circuits are compiled with EPFL's T-256 fork of Noir and translated from ACIR into R1CS by a custom Spartan backend.

Scope

The review covered the following repositories at the commit references listed below:

- `eid-privacy/spartan-backend` at commit `205442b2`: the `c0200_swiyu_jwt` and `c0202_sicpa_backend_constant` age verification circuits, the shared `circuits/eid` helpers, the `preprocessing/c0200_siyu_jwt` witness preprocessor, and the `spartan-backend` and `algebra-utils` Rust crates.
- `eid-privacy/zkp-pocs` at commit `d58bc79d`: the `c06_non_revocation` circuit and the `c09_full_proof` circuit, which combines credential authentication, holder binding, age verification, and non-revocation, together with the non-revocation proposal documented in `noir/README.md`.

The `eid-privacy/noir` T-256 fork at tag `t256-v0.26-1` was originally outside the audit scope but was used to compile the circuits and generate witnesses. Selected parts were consulted to establish the intended ACIR and T-256 semantics. A preliminary review of the delta of this fork with respect to the original repository was performed with the help of our AI tool zkao.

§02

Overview

The reviewed system is designed to let a user prove facts about a digital credential without directly revealing personal information. A trusted issuer signs the credential to authenticate its contents, such as date of birth and the public key associated with their device. The holder can then generate a cryptographic proof that they meet an age threshold (for instance, “I am 25 years old or older”) and possess the corresponding device private key, without disclosing their date of birth or the complete credential. The implementation uses credentials from Switzerland’s swiyu infrastructure, described below, together with Noir circuits, a modified Noir compiler, and a custom ACIR to R1CS backend using Vega, a proving system built on Spartan.

In the following we briefly explain the main architectural modules under review. The stack separates the definition of the proof from its generation for a particular credential. Developers write the required checks as Noir circuits. EPFL’s modified Noir compiler translates those circuits into ACIR, an intermediate representation of their constraints. The Spartan backend then translates ACIR into the constraint system used by the underlying Vega proving library.

For each presentation, the holder’s preprocessing tools read the credential and prepare the private inputs and signature intermediates needed by the circuit. Noir executes the circuit to compute a witness, and the Spartan

backend uses that witness to generate a proof. The verifier checks the proof against the intended circuit and independently selected public inputs.

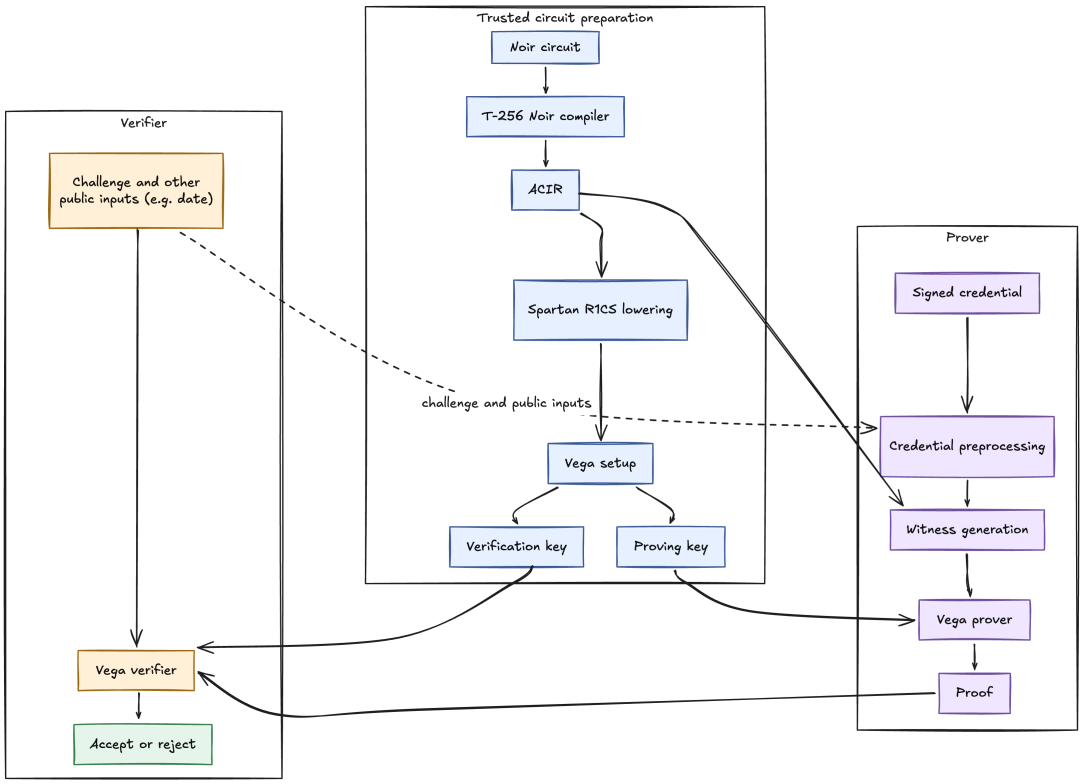


Figure 1: Intended production architecture for the C0200/C0202 T-256, Spartan backend, and Vega proving flow.

The scoped circuits use two distinct proving flows:

CIRCUITS	CREDENTIAL FORMAT	COMPILER AND PROVING BACKEND
`c0200` and `c0202`	swiyu SD-JWT	T-256 Noir, Spartan backend, and Vega
`c06` and `c09`	Custom fixed size credential string	Standard Noir and Barretenberg

Figure 1 and the sections describing the T-256 toolchain and Spartan backend refer to the first flow. The non-revocation section later in this overview describes the separate C06/C09 experiments.

Swiyu credentials and selective disclosure

Swiyu is Switzerland's infrastructure for issuing, holding, and verifying digital credentials, including the Swiss E-ID. Its credential profile uses SD-JWT, or Selective Disclosure JSON Web Token. A JWT carries a set of claims encoded as JSON and authenticated by an issuer's digital signature. SD-JWT allows some claims to be represented by cryptographic hashes in the signed payload. The holder receives the corresponding disclosures, which contain the claim values and random salts, and can reveal selected disclosures for the verifier to check against those hashes.

The current swiyu Beta-ID includes selectively disclosable age threshold claims such as `age_over_18`, and the official verifier example requests only that claim. A simplified payload signed by the issuer can be represented as follows:

```
{
  "iss": "did:tdw:...",
  "_sd_alg": "sha-256",
  "_sd": ["<digest of the age disclosure>"],
  "cnf": {
    "jwk": { "kty": "EC", "crv": "P-256", "x": "...", "y": "..." }
  }
}
```

The holder may then select the corresponding disclosure, shown here after decoding:

```
["random salt", "age_over_18", true]
```

The verifier hashes the encoded disclosure and checks that the result occurs in the signed `_sd` array. It learns only the Boolean value asserted by the issuer, rather than the holder's birth date. A zero-knowledge proof is therefore unnecessary for this predefined majority test. However, this selective disclosure mechanism works only because the issuer calculated `age_over_18` in advance and included it in the documented Beta-ID crede

ntial. It cannot calculate a new predicate from a hidden birth date when the credential is presented and is limited to the thresholds and relationships chosen by the issuer.

The Innosuisse milestone report motivates the need for adding zero-knowledge proofs. Such proofs can evaluate predicates over private attributes without requiring the issuer to precompute every useful relationship, including richer statements such as an age interval. The project also aims to hide values unique to the credential, including the issuer signature and holder device public key, which could otherwise link separate presentations. The reviewed circuits follow this approach by privately checking the signed credential and birth date disclosure and proving an age predicate such as being at least 25. In this stack, the issuer uses a P-256 ECDSA signature, and the credential identifies a device public key used for holder binding.

Credential preprocessing

Before proof generation, a preprocessing step reads the credential and its signatures, performs part of the P-256 arithmetic outside the circuit, and prepares the private and public values used by the prover and verifier.

On the issuer side it recovers the ECDSA point R from the JWT signature and inverts s. The circuit later checks $R = ts^{-1}G + rs^{-1}Q_{\text{iss}}$, where t is the SHA-256 digest of the JWT signing input converted into a circuit field element and $r = R.x$.

On the holder side, the device has signed a verifier challenge with a P-256 ECDSA pair (r, s). Performing the standard verification equation inside the circuit would require several scalar multiplications. The construction instead follows the modified ECDSA verification equation used by Microsoft's Crescent. Starting from $R = s^{-1}(eG + rQ_{\text{dev}})$, multiplying by sr^{-1} and rearranging gives $Q_{\text{dev}} = s(r^{-1}R) - er^{-1}G$. Crescent therefore defines the following two derived points, as can also be seen in its reference implementation:

$$T = R r^{-1}, \qquad U = -e\, r^{-1} G,$$

where e is the challenge digest. This reduces the in circuit P-256 workload to one scalar multiplication and one point addition, moving the inversions and other point calculations into preprocessing and substantially reducing the constraint count. The circuit then checks $x(Ts + U) = x(Q_{\mathrm{dev}})$. T and U are public, so they can be checked against the challenge outside the circuit (s stays private). The recovered R_{dev} is used only to form T on the host.

Age verification circuits

The `c0200_swiyu_jwt` and `c0202_sicpa_backend_constant` circuits are intended to establish three facts in one proof: the issuer authenticated the credential payload, the hidden birth date satisfies the age requirement, and the device associated with the credential approved a verifier challenge. Both circuits keep the payload and birth date private while exposing the issuer key, current date, and values needed for holder binding as public inputs.

First, the circuits authenticate the credential. They reconstruct the JWT signing input from its header and payload, hash it with SHA-256, and check the resulting digest with the issuer's P-256 public key. The preprocessing values R and s^{-1} are used to express this check as the issuer equation described above.

Second, the circuits check the age claim. They rebuild and encode the SD-JWT birth date disclosure `["salt","birth_date","value"]`, hash the encoded disclosure, and require the resulting digest to occur in the authenticated payload. They then interpret the disclosed value as a date in `YYYY-MM-DD` form and compare it with the verifier supplied current date to apply the 25 year threshold.

Third, the circuits perform the holder binding check. They extract the x-coordinate of the device public key from the credential and enforce the Crescent equation $x(Ts + U) = x(Q_{\mathrm{dev}})$. Here, s remains private, while

T and U are public values derived during preprocessing. Complete holder authentication also requires the verifier to check that T and U correspond to its fresh challenge, as discussed later in this report.

The circuits do not parse the complete JSON structure. Instead, the payload is stored in a Noir `BoundedVec`, which has a fixed capacity and a separate logical length, and the prover supplies byte offsets identifying the disclosure digest and device key. This design choice is relevant to several findings below.

The two circuits otherwise enforce the same intended statement but construct the JWT signing input differently. `c0200` fixes the encoded JWT header to `{"typ":"JWT","alg":"ES256"}`. `c0202` instead accepts a public encoded header that also carries the issuer public key.

T-256 Noir toolchain

Noir circuits perform their arithmetic in a finite field. Noir's usual configuration uses the scalar field of BN254, which is approximately 254 bits, and its embedded curve operations target Grumpkin. P-256 coordinates instead belong to a field defined by a different 256-bit prime. They therefore cannot in general be represented canonically as ordinary values in the BN254 field. This makes the usual Noir configuration unsuitable for directly expressing the P-256 operations used by the issuer and device signatures. Solving this usually involves chunking coordinates into smaller limbs, which requires more complex and less efficient arithmetic in-circuit.

EPFL's `eid-privacy/noir` fork adds a `t256` configuration for this purpose. In this configuration, Noir's native `Field` and the ACIR `FieldElement` use the scalar field of Tom-256. Its modulus equals the P-256 coordinate field modulus, so P-256 x and y coordinates can be carried directly by circuit wires without first splitting them into smaller limbs.

P-256 quantity or operation	Mathematical domain	Effect of the T-256 circuit field
Coordinates x, y	P-256 base field $\mathbb{F}_p$	Native: the circuit field is exactly $\mathbb{F}_p$
Point addition and doubling	Formulas over $\mathbb{F}_p$	Efficient: coordinate arithmetic uses ordinary circuit field operations
Scalar k	P-256 scalar field $\mathbb{F}_n$, where $n \neq p$	Not native: scalars require an explicit canonical range and arithmetic modulo n
Scalar multiplication kP	P-256 group operation controlled by $k \in \mathbb{F}_n$	Still requires a custom MSM gadget, scalar bit decomposition, and repeated point operations

T-256 therefore makes the coordinate arithmetic inside P-256 point operations efficient, and indirectly reduces the cost of scalar multiplication, but it does not turn P-256 scalar multiplication into native field multiplication. P-256 scalars remain defined modulo n, not modulo the T-256 scalar modulus p, and must be range constrained and interpreted separately.

The `nargo-t256` compiler is `nargo` built with this configuration. It runs the usual Noir compilation passes and emits ACIR over the new field. Calls to `t256_ec_add` and `t256_msm` become the standard ACIR `EmbeddedCurveAdd` and `MultiScalarMul` opcodes, while the fork's T-256 solver interprets those opcodes as P-256 operations when generating the witness.

The Spartan backend then reads the ACIR and witness, translates the P-256 opcodes into R1CS constraints, and asks Vega to produce the proof. Vega's Hyrax commitment operations use Tom-256 because the scalar field of the commitment curve must match the field of the R1CS. The two curves consequently have different roles. On the one hand, P-256 expresses the

credential signature and holder binding equations. On the other hand, Tom-256 supports the proof system. Correctness depends on the compiler, witness solver, and Spartan lowering all assigning exactly the same meaning to the shared ACIR opcodes.

Spartan backend and Vega proving system

The repository and Rust crate retain the name `spartan-backend`, but proof generation and verification use Vega's `VegaZkSNARK` implementation from the `vega-prover` library. Vega is built on the Spartan proof protocol and incorporates folding and reusable preparation. The custom backend supplies the ACIR lowering and gadgets, while Vega supplies the underlying proof system.

Together, the backend and Vega replace Noir's usual Barretenberg proving step. The backend receives the compiled ACIR program and the witness generated by `nargo-t256`, then translates each supported ACIR operation into an equivalent Bellpepper R1CS constraint over the Tom-256 scalar field. The same synthesis process identifies the public inputs and is run during proof generation and verification before the resulting constraint system is passed to Vega.

The backend implements only the subset of ACIR needed by the reviewed circuits. The supported opcodes are `AssertZero`, `MemoryInit`, and `MemoryOp`, together with `BlackBoxFuncCall` for `RANGE`, `Sha256Compression`, `EmbeddedCurveAdd`, and `MultiScalarMul`. Multi scalar multiplication is limited to one point. Calls to separate constrained ACIR functions and unsupported black boxes, including ACIR's native P-256 ECDSA operation, are rejected. Brillig computations are not translated into R1CS, and their outputs must already be present in the witness and must be constrained by later ACIR operations.

Embedded curve addition and scalar multiplication are handled by custom P-256 gadgets in the black box router. These gadgets implement the curve formulas inside R1CS and bind their calculated coordinates to the ACIR

output witnesses. They are therefore responsible for enforcing all required point membership, scalar, predicate, and point at infinity rules.

The backend integrates Vega's optional prepared proving mode for credentials that are presented repeatedly. The application identifies the inputs that may change for each presentation, such as the device signature scalar and the Crescent points in `c0200`. The backend's dependency analysis divides the circuit into a reusable invariant part and an online part. Vega commits to the invariant witness during preparation and reuses that state with fresh randomness for subsequent proofs, while the backend recomputes the online portion. This improves presentation time, but it also makes correct dependency tracking and protection of the prepared state security sensitive.

Non-revocation proof

The `zkp-pocs` repository explores non revocation using a compact custom credential format rather than the swiyu SD-JWT format described above. Each credential is a fixed length string containing a device key, date of birth, and unique credential identifier at predetermined positions.

The issuer periodically signs a revocation list covering a consecutive range of credential identifiers. The list contains the first identifier in that range, a timestamp, and a bitmap with one bit for each credential. A set bit means that the corresponding credential has been revoked. The list and selected credential remain private, while the verifier supplies the issuer public key and current time.

The standalone `c06_non_revocation` circuit is intended to prove that the credential identifier falls within the selected list, its bit is clear, the public current time is before the list's seven day expiration boundary, and the issuer signed the list. It isolates the non revocation calculation as a proof of concept. It does not authenticate the credential string from which the identifier is read, so by itself it does not establish that the checked identifier belongs to an issuer issued credential held by the prover.

The `c09_full_proof` circuit combines the same list check with three additional checks over the same credential: an issuer signature on the credential, a device signature on the verifier challenge, and an age threshold of 18. This binds the revocation identifier to the authenticated credential and is closer to a complete presentation than `c06`.

Both circuits verify P-256 signatures through Noir's native `std::ecdsa_secp256r1::verify_signature` operation. The reviewed Spartan backend does not implement that ACIR black box, and the custom revocation list is not the status list format used by swiyu.

§03

Summary

The reviewed stack is primarily a research and performance prototype. Its overall approach is technically coherent: the T-256 Noir fork provides a field suitable for representing P-256 coordinates, the Spartan backend translates the resulting ACIR into the Vega proof system, and the prepared proving flow demonstrates a promising way to reduce presentation time. We did not identify a fundamental flaw in this high level architecture. The findings instead concern concrete implementation details and missing validation in the compiler fork, backend, circuits, and verifier integration. Several of these omissions currently affect soundness, and the demonstrated components do not yet form a complete production presentation protocol.

Although known before this review, three limitations are central to its security: the demonstration verifier does not enforce trusted policy or presentation freshness, the T-256 compiler can accept invalid P-256 points during constant folding, and the Spartan backend omits P-256 point membership checks.

The additional implementation findings show further differences between the intended Noir program and the statement ultimately proved. The T-256 compiler can calculate an MSM with several terms incorrectly and embed

that result in the compiled circuit. In the Spartan backend, missing constraints make range checks and scalar decomposition unsound. Its treatment of the point at infinity and other exceptional curve cases can both reject valid operations and produce false MSM results.

The application circuits introduce further weaknesses. In `c0200` and `c0202`, prover supplied offsets can make the age and holder checks read data outside the issuer signed payload. Their issuer signature equation also accepts a zero inverse scalar and the point at infinity once the backend handles infinity correctly. Separately, C06 intentionally isolates the non revocation check for benchmarking; its documentation should clarify that it does not authenticate the credential or holder.

We also found narrower compatibility, completeness, and operational issues. These include a difference between the scalar domains accepted by ACIR and Spartan, incomplete handling of public return values, rejection of some valid issuer signatures, incomplete birth date validation, and sensitive witness data in prepared proving artifacts and debug logs. The detailed findings and discussion below describe the conditions under which each issue matters and distinguish defects in the proved relation from requirements that must be enforced by a production verifier.

§04

Discussion and Recommendations

The findings above concern the relations enforced by the circuits and backend. A complete credential presentation also depends on protocol decisions outside those relations, including who selects each public input, which credential formats are accepted, and how current revocation state is obtained. This section makes those assumptions explicit and describes ways to reduce them as the prototype moves toward production.

Prover and verifier roles

A zero-knowledge proof binds its public inputs to the proved computation, but it does not establish that those inputs came from a trusted source. The relying party must independently select or validate every value that expresses policy, including the circuit verification key, issuer key, credential type, current time, and presentation challenge.

Although known before this review, the demonstration verifier does not enforce this separation. The same host flow that prepares the private witness also writes the public verifier inputs, including the Crescent points T and U derived from `challenge_nonce`. A production integration should instead have the verifier issue a fresh, expiring challenge, derive or validate the corresponding public values, pin the intended circuit and issuer policy, and reject reuse of the same presentation context.

Offset based JWT lookups

How far the JWT circuits go toward a presentation also depends on the assumptions we make about the credentials. In `c0200` / `c0202`, the date of birth and the holder key are not found by parsing the SD-JWT JSON. The prover supplies byte offsets (`dob_sd_offset`, `x_offset`), and the circuit matches a disclosure digest and a `"x":"` coordinate at those positions in the payload. That is a simple lookup using byte offsets rather than a JSON decoder. The present implementation of those lookups is also unsound; see Offsets supplied by the prover allow reads beyond the signed payload. Independently of that bug, the offset design is only as strong as those assumptions:

Level 1: rely on a controlled credential format. The circuit can keep matching bytes at offsets supplied by the prover, without proving their place in the JSON. This is sound only if:

- **Credential structure.** Each accepted payload is one valid JSON object, with no trailing data other than whitespace, and with no repeated property that affects security. Every accepted credential follows a

format and version covered by an explicit compatibility policy (the issuer signature covers `vct`, which these circuits do not check).

- **Date of birth.** Claims supplied by the holder cannot contain arbitrary text or JSON fragments. A matching disclosure digest for the date of birth occurs only as a complete entry of the intended root `_sd` array, not in another claim, inside a longer value, or as a disclosure for a different `birth_date` claim.
- **Selective disclosure.** The issuer only signs credentials that use SHA-256 for selective disclosure. The root `_sd` property is unique, and `_sd_alg` is not set to another algorithm.
- **Holder key.** Every `"x":"` followed by 43 valid base64url characters is the intended holder key coordinate, or the same coordinate repeated. The payload contains no other device, recovery, or extension key with a different `x`. The intended key is a JWK with `kty`, `crv`, `x`, and `y`, 43-character unpadded base64url coordinates of a valid P-256 point, and every copy of the key uses the same coordinates.

We compared these assumptions with Swiyu Generic Issuer at commit `a6d8ec46`. That implementation emits a single JSON object, keeps `cnf`, `vct`, `_sd`, and `_sd_alg` out of subject data, uses SHA-256 disclosures, and stores the verified holder key under `cnf.jwk`. Whether other claims can contain data controlled by the holder still depends on the issuer integration. The same signing key can also cover several credential types. The `c0200` sample stores the holder key under both `cnf` and `cnf.jwk`, while this issuer version uses only `cnf.jwk`. The layout has already changed, so a circuit that uses byte offsets needs a pinned type and version.

Level 2: validate the relevant JSON structure. If those restrictions on the issuer are not something the verifier wants to take as given, the circuit has to prove more about the payload: that `dob_sd_offset` is a complete 43-character element of the unique root `_sd` array, that `_sd_alg` is absent or uniquely `sha-256`, and that the holder key is the unique P-256 JWK at `root.cnf.jwk`, with complete `x` and `y` of a valid point. The scan should reject malformed structure, trailing data, repeated relevant

properties, matches at the wrong nesting level, and incorrectly escaped strings. Supporting the `c0200` layout also means checking that `root.cnf.x` matches `root.cnf.jwk.x`. Colliding bytes in unrelated fields then no longer matter. The circuit still assumes each accepted credential follows one of the supported layouts.

Level 3: parse the complete JSON document and validate its schema. If the same issuer key signs several credential types, or if the JSON layout is not frozen, the circuit parses the logical payload, rejects duplicate property names after decoding JSON escapes, and validates the schema selected by the authenticated `vct`. Claims are identified without fixed byte positions and without restricting unrelated fields. The verifier still pins which issuers and credential types it trusts.

Non-revocation

The C06 and C09 circuits appear to be preliminary experiments rather than components of the T-256 and Spartan flow. The repository describes them as proofs of concept, pins standard Noir `v1.0.0-beta.21` and Barretenberg, and uses `nargo compile` followed by the Barretenberg `bb` commands for proof generation and verification. They do not compile with the T-256 Noir fork or produce proofs through the reviewed Spartan backend.

Integrating non-revocation with the rest of the stack is therefore still an open design problem. At a basic level, `c06` and `c09` are written for a packed credential string and verify signatures with `std::ecdsa_secp256r1::verify_signature`. The Spartan backend has no handler for that black box. The list they consume is also not the `statuslist+jwt` Token Status List that swiyu issues, and `c0200` / `c0202` do not read `status.status_list` from the SD-JWT, so these circuits cannot be checked against the status registry swiyu already runs. C06 intentionally omits credential authentication to measure the non revocation component independently. This benchmark boundary should be documented explicitly; see Standalone non revocation circuit limitations should be documented.

For a presentation that includes both age and non-revocation to be sound, those checks also have to be about the same credential. One option is a single circuit, as `c09` already does for the packed format. Another is to keep separate proofs and have the SD-JWT circuit extract the identifier from the signed payload, under the same kind of layout assumptions as the date of birth and the holder key, and share it with the revocation circuit, for example as a public output or as a commitment both proofs open.

Finally, the signed list should identify the context in which it is valid. Its current signed contents do not include a format version, registry identifier, credential type, status purpose, deployment identifier, or epoch. Without those fields, safe use assumes that the same signing key and credential identifier namespace are never reused across incompatible contexts. The mechanism used to distribute the latest state should provide this context and freshness without requiring a presentation time query that reveals the holder's credential range or identifier.

Noir compiler and fork assurance

Noir is a sophisticated compiler with a substantial testing strategy, including unit and integration tests, differential fuzzing, and tests for individual optimization passes. Parts of the compiler have also received an internal security review, and the public issue tracker records at least one compiler finding from an external Veridise review. Formal verification tools are also emerging for selected components, including Noir's SSA verification artifacts, NAVe's analysis of compiled ACIR, and Lampe's Lean model of Noir programs. Noir also has meaningful public adoption: at the time of this review its GitHub repository had approximately 1,400 stars and 400 forks, and its official ecosystem page listed projects using Noir across payments, infrastructure, and other applications.

These indicators support an assessment of moderate engineering maturity, but popularity, testing, and partial reviews do not prove that every compilation path preserves the relation expressed by the source program. Noir's own security policy states that it is not fully audited or recommended for production. Its security assurance should therefore be considered below

that of a production hardened or formally verified compiler, particularly for identity and authorization. The exact compiler version and generated ACIR remain part of the trusted computing base.

This assurance does not transfer automatically to the T-256 fork, which changes the circuit field and adds fork specific point operations, solver behavior, and constant folding paths. In our preliminary review of the fork's delta from upstream Noir, we identified one High severity compiler issue: constant folding can calculate an MSM with several terms incorrectly, which can cause soundness failures in circuits that use several points with different scalars. The reviewed C0200 and C0202 circuits use only single point MSMs and are not affected by this particular issue. We recommend a deeper audit of the complete fork delta, including the interaction of its changes with compile time evaluation, optimization, witness generation, validation, and Spartan lowering.

Recommended next steps

Before moving toward production, the project should resolve the reported findings and Known Issues across the complete stack. We also recommend another independent audit once the circuits and surrounding protocol are close to their production design. In particular, the prover and verifier integration, non-revocation design, Spartan backend, and Noir fork will be important areas to audit given the project's current prototype status.

§05

Findings

Below are listed the findings found during the engagement. High severity findings can be seen as so-called "priority 0" issues that need fixing (potentially urgently). Medium severity findings are most often serious findings that have less impact (or are harder to exploit) than high-severity findings. Low severity findings are most often exploitable in contrived scenarios, if at all, but still warrant reflection. Findings marked as informational are general comments that did not fit any of the other criteria.

HIGH

#00 Prover-controlled offsets allow reads beyond the signed payload

`circuits/c0200_swiyu_jwt/src/main.nr, circuits/c0202_sicpa_backend_constant/src/main.nr`

#01 Zero inverse scalar and infinity point bypass issuer authentication

`circuits/c0200_swiyu_jwt/src/main.nr, circuits/c0202_sicpa_backend_constant/src/main.nr`

#02 Range decomposition values are not constrained to be Boolean

`spartan-backend/src/noir/synthesis/blackbox/range.rs, spartan-backend/src/noir/synthesis/blackbox/router.rs`

#03 Noncanonical scalar decompositions change MSM results

`spartan-backend/src/noir/synthesis/allocated_point.rs`

#04 An infinity and zero x coordinate collision produces false MSM results

`spartan-backend/src/noir/synthesis/allocated_point.rs`

#05 Point at infinity handling breaks valid curve operations

`spartan-backend/src/noir/synthesis/allocated_point.rs`

#06 Constant folding compiles P256 MSMs to incorrect results

`noir/acvm-repo/t256_blackbox_solver/src/lib.rs`

MEDIUM

#07 Fixed base MSM offset collisions can produce incorrect results

`spartan-backend/src/noir/synthesis/allocated_point.rs`, `spartan-backend/src/noir/synthesis/constant_point.rs`, `spartan-backend/src/noir/synthesis/blackbox/multi_scalar_multiplication.rs`

#08 Missing digest reduction prevents some valid issuer signatures from being verified

`circuits/eid/src/field_utils.nr`, `circuits/c0200_swiyu_jwt/src/main.nr`, `circuits/c0202_sicpa_backend_constant/src/main.nr`

#09 MSM accepts scalars outside the canonical P256 domain

`spartan-backend/src/noir/synthesis/blackbox/multi_scalar_multiplication.rs`

#0a Public return values are treated as private witnesses

`spartan-backend/src/noir/circuit_reader/named_parameters_mapping.rs`, `spartan-backend/src/noir/circuit_reader/prover_input_mapping.rs`

LOW

#0b Birth date parsing and age checks do not enforce calendar semantics

`circuits/c0200_swiyu_jwt/src/main.nr`, `circuits/c0202_sicpa_backend_constant/src/main.nr`, `noir/c09_full_proof/src/main.nr`

#0c Prepared proving state and debug logs expose private witness data

`spartan-backend/src/precompute.rs`, `spartan-backend/src/circuit_instance.rs`, `spartan-backend/src/lib.rs`

INFORMATIONAL

#0d Standalone non revocation circuit limitations should be documented

`noir/c06_non_revocation/src/main.nr`

#0e Partial base64url chunks use bytes beyond the logical payload

`circuits/eid/src/fast_base64.nr`

KNOWN ISSUE

#0f Curve operations do not enforce P256 point membership

spartan-backend/src/noir/synthesis/blackbox/ec_add.rs, spartan-backend/src/noir/synthesis/blackbox/multi_scalar_multiplication.rs

#10 The demonstration verifier does not enforce policy or presentation freshness

preprocessing/c0200_siyu_jwt/src/main.rs, spartan-backend/src/circuit_instance.rs, spartan-backend/src/nizk_verifier.rs, noir/scripts/server/verifier-server.py

#11 Unchecked P256 points can be folded into false curve statements

noir/acvm-repo/t256_blackbox_solver/src/lib.rs

HIGH · #00

Prover-controlled offsets allow reads beyond the signed payload

circuits/c0200_swiyu_jwt/src/main.nr,
circuits/c0202_sicpa_backend_constant/src/main.nr

Description. The payload input in both circuits is a `BoundedVec<u8, PAYLOAD_MAX_LEN>` with a backing array of fixed capacity and a separate logical length. The circuit verifies the issuer's signature using the logical payload, whose length is given by `payload.len()`. The later checks for the date of birth and the holder's public key instead read from the entire backing array returned by `payload.storage()`.

For example, the provided input for `c0202_sicpa_backend_constant` declares a payload length of 498 bytes, but its backing array holds 2,048 bytes. The issuer's signature does not cover the remaining 1,550 bytes.

The prover also supplies `dob_sd_offset` and `x_offset`. The circuit uses them without checking that the complete reads remain within the logical payload:

```
for i in 0..COORD_BASE64_LEN {
    assert(
        payload_storage[dob_sd_offset + i] == expected_digest[i],
        "DOB disclosure digest mismatch",
    );
}

let device_key_x_bytes =
    extract_coord_for_key(payload_storage, x_offset, ASCII_X);
```

The first operation reads 43 bytes and compares them with the base64url digest of the date-of-birth disclosure supplied by the prover. The second looks for the five bytes `"x":"` and treats the following 43 bytes as the `x` coordinate of a public key. `dob_sd_offset` points to the first digest character, while `x_offset` points to the opening quote of the `x` property.

A malicious prover can leave the payload and the issuer's signature unchanged, place a chosen disclosure digest and public key coordinate after the logical end of the payload, and set both offsets to those values. The preprocessing code does not prevent this attack because a malicious prover can bypass it and construct the circuit witness directly.

Impact. A prover with a credential signed by the issuer can satisfy the age check using a date of birth that the issuer did not authenticate. The prover can also substitute a different public key and use it to pass the proof of possession, breaking the intended binding between the credential and the holder's device.

Recommendation. The appropriate fix depends on which credential layout assumptions the project is willing to retain and the corresponding efficiency tradeoff. Offset based JWT lookups discusses options ranging from bounded byte matching to validation of the relevant JSON structure or complete parsing. At minimum, both prover supplied offsets must be constrained so that the complete reads remain within the logical signed payload length. The check for `x` must cover the five bytes `"x":"`, the 43 bytes of the coordinate, and the closing quote, for 49 bytes in total.

Client response. The client addressed this finding in PR #65 by adding bounds checks for `dob_sd_offset` and `x_offset` in C0200 and C0202. The implementation continues to use byte matching at offsets supplied by the prover. See Offset based JWT lookups for details on the credential layout assumptions supporting this approach and the security and efficiency tradeoffs of adding structural validation or a complete JSON parser.

HIGH · #01

Zero inverse scalar and infinity point bypass issuer authentication

circuits/c0200_swiyu_jwt/src/main.nr,
circuits/c0202_sicpa_backend_constant/src/main.nr

Description. Both reviewed circuits verify the issuer signature using private values `R_jwt` and `s_inv_jwt` supplied by the prover. They enforce the equation

$$R_{jwt} = s^{-1}tG + s^{-1}rQ_{issuer},$$

where the circuit defines r as the x coordinate of `R_jwt`. However, neither circuit requires `s_inv_jwt` to be a nonzero canonical P256 scalar or requires `R_jwt` to be different from the point at infinity. See the issuer equation in `c0200_swiyu_jwt` and `c0202_sicpa_backend_constant`.

A malicious prover can choose

```
s_inv_jwt = 0
R_jwt = (0, 0) = infinity
```

Both scalar multiplications on the right then produce the point at infinity, independently of the signing digest and the issuer public key. Since `(0,0)` is the canonical T256 coordinate encoding of infinity, the final coordinate equality succeeds. The resulting issuer check no longer authenticates the payload.

On the unmodified backend, this attack is accidentally blocked by the separate completeness defect described in Point at infinity handling breaks valid curve operations. The backend incorrectly reconstructs infinity as a finite point during chained operations and fails before producing the proof. To isolate the security issue, we temporarily patched that completeness defect without adding curve membership constraints. With this diagnostic

patch applied, our proof of concept generated and verified a Vega proof through the Spartan backend using `s_inv_jwt = 0` and `R_jwt = (0,0)`. We shared the proof of concept and temporary patch with the client.

This issue is independent of the missing P256 curve membership constraints. The affine pair `(0,0)` is not a finite point on P256, but it is the designated coordinate encoding of the valid group identity and must be interpreted as such. Correcting the backend's infinity handling therefore exposes the missing circuit validation rather than causing it.

The circuit also fails to establish the complete ECDSA range requirement for r. ECDSA requires $1 \leq r < n$, where n is the P256 group order. If $r = 0$, the issuer key term disappears:

$$R_{jwt} = s^{-1}tG + s^{-1}rQ_{issuer} = s^{-1}tG.$$

The equation then no longer depends on the issuer public key. We did not identify a separate practical attack that exploits this missing check, but it is another reason the current relation does not fully enforce ECDSA semantics.

Impact. After correct infinity support is implemented, a malicious prover can satisfy the issuer equation without a valid issuer signature. The prover can choose an arbitrary payload, disclosure, and holder key while retaining the verifier selected issuer public key, then generate a proof that the circuit treats as an authenticated credential presentation. The current accidental failure is not a valid security control because fixing the independent completeness defect activates the bypass.

Recommendation. Require `s_inv_jwt` to be a canonical nonzero P256 scalar by enforcing $1 \leq s_{inv,jwt} < n$. Require `R_jwt` to be a valid finite P256 point rather than infinity. Derive r according to ECDSA semantics, reduce the x coordinate modulo n where required, and enforce $1 \leq r < n$. The backend should independently enforce the canonical scalar range for every MSM input, while the circuit must impose the protocol specific nonzero requirements for issuer signature values.

Client response. The client addressed the demonstrated bypass in PR #65 by requiring `s_inv_jwt` and both coordinates of `R_jwt` to be nonzero in C0200 and C0202. These checks reject the zero inverse scalar and infinity witnesses used in the attack. The upper bound `s_inv_jwt < n` remains to be enforced through the backend fix for MSM accepts scalars outside the canonical P256 domain.

HIGH · #02

Range decomposition values are not constrained to be Boolean

spartan-backend/src/noir/synthesis/blackbox/range.rs, spartan-backend/src/noir/synthesis/blackbox/router.rs

Description. The Spartan backend translates every positive width ACIR `RANGE(x, n)` opcode into an underconstrained R1CS relation. The translation allocates each intended bit b_i as an ordinary field element rather than a Boolean variable.

`field_into_allocated_bits_le` derives zero or one assignments when it generates an honest witness, but uses `AllocatedNum::alloc` for each value:

```
let bits = values
    .into_iter()
    .rev()
    .take(bit_size)
    .map(|b| optional_boolean_to_ff(b))
    .enumerate()
    .map(|(i, b)| {
        AllocatedNum::alloc(
            cs.namespace(|| format!("bit {} of {}", i, witness_index)),
            || optional_ff_to_result(b),
        )
    })
    .collect::<Result<Vec<_>, SynthesisError>>()?;
```

This allocation does not constrain b_i to zero or one. The `RANGE` router subsequently enforces only

$$x = \sum_{i=0}^{n-1} b_i 2^i.$$

A malicious prover may assign arbitrary field values to the purported bits. For example, an 8 bit range check for $x = 256$ is satisfied by setting $b_7 = 2$ and every other $b_i = 0$. More generally, for any positive width the

prover can set $b_0 = x$ and all remaining values to zero, making the range check vacuous.

We demonstrated this issue with a proof of concept that constructs a real ACIR `RANGE` opcode and passes it through the production `NoirCircuitSynthesizer`. The honest decomposition for $x = 256$ and an 8 bit range is unsatisfied. Replacing the auxiliary assignment with $b_7 = 2$ makes the generated constraint system satisfied. We shared this proof of concept with the client. It directly confirms the faulty production lowering, although it does not create and verify a complete Vega proof.

The reviewed circuits rely on fixed width integer semantics and comparisons that Noir lowers using range constraints. In particular, the age assertions in `c0200_swiyu_jwt` and `c0202_sicpa_backend_constant` use an unsigned greater than or equal comparison. The soundness of the arithmetic and comparison constraints depends on the values used as bits actually being Boolean.

Impact. An adversarial SNARK prover can satisfy range checks with values outside the interval $[0, 2^n)$. This breaks the backend's translation of Noir fixed width integers, bytes, comparisons, and arithmetic checks. It therefore undermines the soundness of the reviewed age predicates and may allow an underage holder to satisfy the age comparison. The current proof of concept confirms the generic range bypass; it does not independently demonstrate a complete credential proof that bypasses the age check.

The normal witness generator assigning genuine bits does not mitigate the issue. A sound R1CS must reject every invalid assignment, including auxiliary assignments chosen directly by a malicious prover.

Recommendation. Allocate each decomposition value as a Boolean variable, or add a constraint

$$b_i(b_i - 1) = 0$$

for every b_i. Retain the existing weighted recomposition constraint.

Client response. The client addressed this finding in PR #65 by replacing `AllocatedNum` with `AllocatedBit` for the decomposition values, enforcing Booleanity while preserving the weighted recomposition constraint.

HIGH · #03

Noncanonical scalar decompositions change MSM results

spartan-backend/src/noir/synthesis/allocated_point.rs

Description. Both the variable base and fixed base scalar multiplication gadgets decompose an allocated scalar using `AllocatedNum::to_bits_le`. See `scalar_mul` and `scalar_mul_fixed_base`.

The decomposition produces Boolean values b_i and enforces only

$$\sum_i b_i 2^i = s \quad \left(\mathrm{mod}\ \ p\right),$$

where p is the modulus of the R1CS circuit field. It does not require the integer represented by the bits to be smaller than p. When both k and $k + p$ fit in 256 bits, they are therefore valid decompositions of the same circuit field value k.

The scalar multiplication gadgets then consume these bits as an integer when computing a point on P256, whose group order is $n \neq p$. The two decompositions consequently produce

$$kG \quad \text{and} \quad (k+p)G,$$

which are generally different points because p is not a multiple of n. Each invocation creates its own decomposition, so a malicious prover can choose a different noncanonical representative for every MSM scalar while keeping the allocated circuit field values unchanged.

Our proof of concept sets the allocated scalar to zero but assigns its decomposition bits to the 256 bit representation of p. The decomposition constraint remains satisfied because $p = 0$ in the circuit field. The fixed base gadget then computes the finite point pG, even though the canonical

scalar value is zero and $0G$ is infinity. The proof of concept passes the real ACIR `MultiScalarMul` operation through the production `NoirCircuitSynthesizer` and confirms that the unchanged R1CS is satisfied after replacing the private decomposition and dependent gadget assignments. We shared this proof of concept with the client. It demonstrates satisfaction of the production constraint system, although it does not create and verify a complete Vega proof.

Impact. A malicious prover can make an MSM return a point that is not the scalar multiplication specified by its ACIR scalar witness. This is a direct soundness failure in the ACIR to R1CS translation.

The reviewed age verification circuits use these MSM gadgets in their issuer and holder signature equations. Since every invocation can interpret its scalar through an independently chosen noncanonical decomposition, the issue may allow those equations to be satisfied without the corresponding P256 relation. We have not constructed a complete credential proof that exploits this freedom, so the precise application level impact remains unconfirmed.

Recommendation. Replace both uses of `to_bits_le` with `to_bits_le_strict`. The strict method also enforces that the represented integer is smaller than the circuit field modulus p, eliminating alternative representatives such as $k + p$. This change does not replace the separate recommendation described in MSM accepts scalars outside the canonical P256 domain. Every general MSM scalar must also satisfy $0 \leq s < n$, while ECDSA values that must be invertible require $1 \leq s < n$.

Client response. The client addressed this finding in PR #65 by using `to_bits_le_strict()` in both scalar multiplication paths, ensuring that the decomposition represents the canonical circuit field value.

HIGH · #04

An infinity and zero x coordinate collision produces false MSM results

spartan-backend/src/noir/synthesis/allocated_point.rs

Description. The shared `AllocatedPoint::add` gadget compares point coordinates before giving precedence to the internal infinity flags.

The function first computes `equal_x` and `equal_y` from the coordinates. It computes an infinity aware addition result through `add_internal`, but its final selection gives `equal_x` priority. When the x coordinates are equal and the y coordinates differ, it selects infinity as if the operands were inverse finite points. This rule is incorrect when one operand is already infinity.

Let P_0 be the valid finite P256 point

```
P0.x = 0
P0.y = 0x66485c780e2f83d72433bd5d84a06bb6541c2af31dae871728bf856a174f93f4
```

For the canonical internal infinity value $(0, 0, 1)$ and $P_0 = (0, y, 0)$, `equal_x` is true and `equal_y` is false. The gadget therefore returns infinity instead of enforcing

$$\infty + P_0 = P_0.$$

This state is reachable during valid variable base scalar multiplication. Our proof of concept supplies a valid point P and the canonical scalar $s = 2^{254}$ such that

$$sP = P_0.$$

In `scalar_mul`, the lower 254 scalar bits are zero. Removing the initial ladder slack produces a correctly tagged internal infinity point. Processing the next bit then calls `AllocatedPoint::add` to compute $\infty + P_0$. Because both x coordinates are zero, the faulty selection returns infinity.

The proof of concept establishes that P and P_0 are valid finite P256 points, the scalar is nonzero and smaller than the P256 group order, and the T256 ACVM solver returns the correct result P_0. Supplying P_0 as the output makes the production Spartan constraint system unsatisfied, while supplying the false infinity result `(0,0)` satisfies the unchanged `MultiScalarMul` lowering. The gadget's normal witness assignments produce this result; the attack does not require invalid points, noncanonical scalar bits, or modified auxiliary assignments. We shared the proof of concept with the client.

The same addition gadget is used by ordinary EC addition. A direct PoC for $\infty + P_0$ through that handler is currently obstructed by the separate completeness problem described in Point at infinity handling breaks valid curve operations, because the handler reconstructs `(0,0)` with the wrong infinity flag. Correcting that boundary behavior would expose this second error in the shared addition gadget. Ordinary finite additions without this infinity and zero x coordinate collision are unaffected.

Impact. The Spartan backend accepts a false scalar multiplication result for valid P256 inputs. A proof can therefore attest that $2^{254}P$ is infinity even though correct P256 arithmetic gives the nonidentity point P_0. This is a direct soundness failure in the ACIR to R1CS translation and justifies High severity even though no separate exploit against the reviewed credential equations has been demonstrated.

Recommendation. Handle infinity before every finite coordinate exception. If only the first operand is infinity, return the second operand. If only the second operand is infinity, return the first. If both are infinity, return canonical infinity. Only when both operands are finite should equal coordinates select doubling or the inverse point case.

Client response. The client addressed this finding in PR #65 by restricting coordinate equality branches to finite operands and giving infinity handling precedence when selecting the addition result.

HIGH · #05

Point at infinity handling breaks valid curve operations

spartan-backend/src/noir/synthesis/allocated_point.rs

Description. The scoped T256 ACIR interface represents the point at infinity using the canonical coordinates $(0, 0)$, whereas the Spartan backend internally represents a point as $(x, y, \texttt{is_infinity})$. The backend does not preserve this distinction consistently across ACIR opcode boundaries.

For example, let P be a valid P256 point. When computing $P + (-P) = \infty$, the Spartan addition gadget correctly produces an internal `AllocatedPoint` with $(x, y, \texttt{is_infinity}) = (0, 0, 1)$. However, ACIR exposes only the two output coordinates. If this result is passed into another operation, `unwrap_point` reconstructs $(0, 0)$ with `is_infinity = 0` and treats it as a finite point. Consequently, the valid expression

```
P + (P + (-P)) = P + infinity = P
```

fails while evaluating the affine formulas because the backend attempts to invert a zero denominator.

The variable base MSM handler reconstructs points in the same way. Although $0 \cdot P$ correctly produces infinity internally, passing its coordinate result into another operation causes the valid expression $7 \cdot (0 \cdot P) = 7 \cdot \infty = \infty$ to fail.

There is an additional issue specific to variable base scalar multiplication. `scalar_mul` converts its input into `AllocatedPointNonInfinity` and evaluates an incomplete affine ladder of point doublings and additions before selecting infinity if the original input was infinity. The final selection does not disable constraints emitted by the unused branch. The first doubling of $(0, 0)$ therefore imposes

$$\lambda(2 \cdot 0) = 3 \cdot 0^2 - 3,$$

which reduces to the impossible constraint $0 = -3$. The constraint system is already unsatisfied before the final selector can return infinity.

We demonstrated these issues with PoCs covering $P + (-P) = \infty$, $\infty + P = P$, $0 \cdot P = \infty$, and $7 \cdot \infty = \infty$. We shared these PoCs with the client.

Impact. Valid P256 computations accepted by Noir and ACVM can fail during Spartan synthesis or produce an unsatisfied constraint system. Any circuit that passes the result of an addition or scalar multiplication that produces infinity into a later curve operation can therefore fail to generate a proof (completeness issue).

Recommendation. Decode ACIR points represented only by coordinates by constraining `is_infinity` to be true if and only if both coordinates are zero, and use this decoder consistently in EC addition and scalar multiplication. Prevent incomplete affine formulas from being evaluated on infinity, either by selecting a constrained safe finite dummy point before evaluating the incomplete ladder and selecting canonical infinity afterward, or by implementing complete group operations.

Client response. The client addressed this finding in PR #65 by consistently decoding `(0,0)` as infinity in EC addition and MSM, and using a finite dummy point during the incomplete scalar multiplication ladder, selecting canonical infinity afterward when the input is infinity.

HIGH · #06

Constant folding compiles P256 MSMs to incorrect results

noir/acvm-repo/t256_blackbox_solver/src/lib.rs

Description. The T256 Noir fork's MSM solver uses `scalars_lo[0]` for every point instead of selecting the scalar corresponding to each term. For an MSM over the points `[G, 2G]` with scalars `[2, 3]`, the expected result is

$$2G + 3(2G) = 8G.$$

The solver instead applies the first scalar to both points and computes

$$2G + 2(2G) = 6G.$$

This affects both runtime witness computation and compiler evaluation. During constant folding, the compiler calls the same solver and replaces a fully constant MSM with the resulting coordinates. The generated ACIR therefore contains the incorrect result and no longer contains the original MSM opcode. Our proof of concept demonstrates this behavior for the example above.

The Spartan backend requires exactly one point and one scalar, panicking during synthesis if an MSM contains more than one term. However, this check cannot detect an operation that constant folding has already removed.

Impact. The compiler can produce a circuit enforcing a different P256 result from the one specified by the Noir source. A backend can correctly prove the generated ACIR while the proof does not establish the intended source statement.

The reviewed `c0200` and `c0202` circuits are unaffected by this particular defect because their MSM calls contain only one point and one scalar. The Noir fork was treated as a dependency outside the formal audit scope; this observation arose from examining its T256 changes.

Recommendation. Select the corresponding low scalar for each point using `scalars_lo[i / 2]` in the loop over coordinate pairs, and validate every scalar independently. Alternatively, reject MSM calls with more than one term in the shared T256 solver. Such rejection must occur before compiler evaluation and constant folding so that invalid operations cannot be replaced with constants before reaching the backend.

Client response. The client addressed this finding in Noir `t256-v0.26-2` by selecting `scalars_lo[i / 2]` for each point and checking every low scalar against the P256 group order. PR #65 updates the backend's toolchain configuration to use this version.

MEDIUM · #07

Fixed base MSM offset collisions can produce incorrect results

spartan-backend/src/noir/synthesis/allocated_point.rs, spartan-backend/src/noir/synthesis/constant_point.rs, spartan-backend/src/noir/synthesis/blackbox/multi_scalar_multiplication.rs

Description. The Spartan backend selects a custom fixed base multiplication path when an MSM uses a constant base and a witness scalar: `multi_scalar_multiplication.rs`. This path derives a deterministic offset point Z and constructs table entries of the form

$$D_j = 4^j B, \qquad K_j = 2^j Z, \qquad T_j[d] = dD_j + K_j.$$

The selected table entries are accumulated using an incomplete affine addition formula: `allocated_point.rs`. The implementation assumes that the accumulator and selected table point will not have the same x coordinate. That assumption is not enforced and does not hold for every valid constant base.

For example, choose the valid P256 base $B = Z/3$, where Z is the point deterministically derived for that base, and use scalar $s = 3$. The first two selected entries are then equal:

$$T_0[3] = 3B + Z = 2Z,$$

$$T_1[0] = 2Z.$$

The incomplete addition receives the same point twice. Its slope constraint

$$\lambda(x_2 - x_1) = y_2 - y_1$$

reduces to $0 = 0$ instead of enforcing the doubling slope. The slope and resulting point can therefore be assigned values that satisfy the generated R1CS but do not equal the correct result $3B$.

The proof of concept in `pocs/fixed_base_offset_collision` exercises this behavior through the production ACIR to R1CS MSM dispatcher and compares the accepted output with the correct result from the T256 solver. It uses a valid P256 base and a canonical scalar.

Impact. For a circuit containing an affected constant base, a malicious prover can produce a proof for a scalar multiplication result that is false under P256 arithmetic. This is a soundness issue in the generic Spartan translation. No exploit was demonstrated against C0200 or C0202 because those circuits use the standard P256 generator, for which no useful relationship with the derived offset point is known.

Recommendation. Use complete constrained group addition when accumulating the fixed base table entries, including correct handling of equal points, inverse points, and infinity. Alternatively, restrict this optimization to an explicit set of audited bases for which all required preconditions are established. An unenforced assumption about the discrete logarithm relationship between the base and the deterministic offset is not sufficient for a generic ACIR lowering.

Client response. The client is evaluating how best to address this finding.

MEDIUM · #08

Missing digest reduction prevents some valid issuer signatures from being verified

circuits/eid/src/field_utils.nr, circuits/c0200_swiyu_jwt/src/main.nr, circuits/c0202_sicpa_backend_constant/src/main.nr

---

Description. Both circuits compute the SHA-256 digest of the JWT signing input and convert the resulting 32 bytes into a circuit field element using `array_to_field`:

```
pub fn array_to_field(arr: [u8; 32]) -> Field {
    let mut field_value: Field = 0;
    for i in 0..32 {
        field_value = field_value * 256 + arr[i] as Field;
    }
    field_value
}
```

```
let signing_digest: [u8; 32] =
    sha256_var(signing_input_storage, signing_input_len);
let t_jwt = T256Scalar {
    hi: 0,
    lo: array_to_field(signing_digest),
};
```

The circuit field modulus p is slightly smaller than 2^{256}. Since the operations in `array_to_field` take place in this field, a digest $d \geq p$ is converted to $d - p$ rather than preserving its 256-bit integer value.

ECDSA instead interprets the digest modulo the P-256 group order n. The preprocessing code follows this behavior when calculating `R_jwt` and `s_inv_jwt`. For a digest at least p, the circuit therefore uses $d - p$, while the preprocessor and the issuer signature use $d \bmod n$. These values differ because p and n are distinct moduli, causing the signature equation in the circuit to fail.

There is also a smaller interval between n and p. Digests in this interval fit in the circuit field, but the T256 solver rejects them because they are not canonical P-256 scalars. Consequently, the normal proving process fails for every digest $d \geq n$.

Assuming that SHA-256 outputs are uniformly distributed, this occurs with probability

$$\frac{2^{256} - n}{2^{256}} \approx 2^{-32}.$$

The other reviewed uses of `array_to_field` convert valid P-256 coordinates, which are already smaller than p. The issue applies to unrestricted 32-byte values such as the SHA-256 digest.

Impact. A holder cannot generate a proof for an otherwise valid credential when its signing digest falls in the affected range. Retrying proof generation does not help because the digest is determined by the credential's signing input. No method of using this behavior to make an invalid issuer signature pass was identified.

Recommendation. Convert the digest to its canonical value modulo the P-256 group order before representing it as a circuit field element. One way to do this is to compare the digest bytes with n and conditionally subtract n before calling `array_to_field`. Since a SHA-256 digest is smaller than 2^{256} and $2n > 2^{256}$, at most one subtraction is required. Apply the same reduction in both the circuit and preprocessing code.

Client response. Fixed. The client addressed the digest reduction in PR #65 by reducing the signing digest modulo the P-256 group order before using it as a scalar. While implementing the fix, the client identified two additional issues. The `Field::lt` comparison used assumptions specific to BN254 and was corrected in the updated Noir fork. Separately, `R_jwt.x` can fall in the interval $[n, p)$ and be rejected as a noncanonical scalar. The client left this coordinate unreduced because the estimated probability is

approximately 2^{-130} for ordinary signatures. We nevertheless recommend reducing `R_jwt.x` modulo n for completeness, so that this rare case does not prevent a valid signature from being verified.

MEDIUM · #09

MSM accepts scalars outside the canonical P256 domain

spartan-backend/src/noir/synthesis/blackbox/multi_scalar_multiplication.rs

Description. The T256 ACIR solver defines the low MSM scalar limb as a canonical P256 scalar. It rejects every value greater than or equal to the P256 group order, denoted by n.

The Spartan MSM lowering requires the high scalar limb to be zero but does not constrain the low limb to be smaller than n. The circuit field modulus p is larger than n, so the backend can represent and accept scalar values in the interval $[n, p)$ that the pinned ACIR solver rejects.

For example, a malicious prover can supply the circuit field value n. This value is nonzero in the circuit field, but scalar multiplication on P256 gives

$$nP = \infty$$

for every finite P256 point P. The backend can therefore accept an MSM with scalar n and infinity output, even though the same invocation is invalid under the pinned ACIR scalar domain.

Our proof of concept compares both implementations using scalar n, the P256 generator, and the canonical infinity output `(0,0)`. The production Spartan constraint system is satisfied, while the T256 solver rejects the same scalar before performing the multiplication. We shared this proof of concept with the client.

This issue is separate from noncanonical scalar decompositions. That finding allows the bits consumed by a scalar multiplication gadget to represent $k + p$ while the allocated circuit field value remains k. Here the allocated value itself lies in $[n, p)$ and the missing constraint is the P256 scalar range required by ACIR.

Impact. The Spartan backend proves a strictly weaker relation than the pinned ACIR semantics by accepting MSM invocations that ACIR defines as invalid. Values $n + k < p$ behave in the P256 group like k, which creates scalar aliases in any circuit that gives the unreduced value independent meaning.

This also affects protocol specific nonzero checks. In the issuer signature equation, checking only `s_inv_jwt != 0` would still permit `s_inv_jwt = n`, which is nonzero as a circuit field value but acts as zero in P256 scalar multiplication. It could therefore reproduce the behavior described in Degenerate issuer signature witnesses bypass authentication. No separate application exploit using this scalar domain mismatch was demonstrated.

Recommendation. Constrain every MSM low scalar limb to satisfy $0 \leq s < n$ before either the fixed base or variable base scalar multiplication path uses it. Reject a constant input outside this range and add a sound comparison constraint for a witness input. Keep the existing requirement that the high limb equals zero. Protocol code must impose any stronger requirements separately. General MSM permits zero, while an ECDSA inverse scalar must satisfy $1 \leq s < n$. This range constraint is distinct from the strict decomposition requirement described in Noncanonical scalar decompositions change MSM results. The range check ensures that the ACIR scalar is smaller than n, whereas strict decomposition prevents the MSM gadget from interpreting that scalar using alternative bits representing $s + p$.

Client response. The client has acknowledged this finding and is working on a fix.

MEDIUM · #0A

Public return values are treated as private witnesses

spartan-backend/src/noir/circuit_reader/named_parameters_mapping.rs, spartan-backend/src/noir/circuit_reader/prover_input_mapping.rs

---

Description. ACIR defines both public parameters and return values as public inputs that the verifier must receive. This requirement is documented directly in the `Circuit` definition.

The Spartan input mapping does not preserve this rule. `map_wires` maps ABI function parameters only to `function.public_parameters` or `function.private_parameters`. It does not map the ABI return type or consume `function.return_values`.

When the solved witness contains a return witness that is absent from this parameter mapping, the prover input mapper silently assigns `public: false`:

```
let wire = match flat_wires.get(w_id) {
    Some(Wire { public, witness }) => Wire::new(*public, *witness),
    None => Wire {
        public: false,
        witness: *w,
    },
};
```

The `public_values` method builds the proof transcript by filtering this incomplete mapping for witnesses marked public. The return witness is therefore enforced by the circuit constraints but omitted from the public input vector supplied to the verifier.

Our proof of concept constructs an ACIR circuit that enforces `returned = private` and declares `returned` in `Circuit.return_values`. It supplies the return witness only to the prover, marks it private in Spartan's input

mapping, and gives the verifier no expected return value. The normal `prove_circuit` and `verify_circuit` entry points nevertheless create and accept the proof. We shared this proof of concept with the client.

Consequently, a circuit that returns an authorization decision proves only that some private result exists. The verifier cannot require the returned decision to equal the expected public value. The reviewed `c0200` and `c0202` circuits do not return values and are not directly affected, but the generic backend does not correctly support circuits that rely on public returns.

There are two related correctness issues in the same mapping and allocation flow:

- The ABI mapper uses `.take(width)` without checking that it received exactly the declared number of witnesses. If an ABI declares a public two byte array but ACIR provides only one public witness, the mapper silently creates a one wire mapping and accepts the truncated public input shape. This mainly affects malformed artifacts or compiler regressions because a trusted verifier normally uses the same pinned artifact.
- The partitioner constructs its witness universe only from opcode reads and writes. A declared public parameter or return that no opcode references may therefore be absent when synthesis attempts to expose it as a public R1CS input. Synthesis then returns `AssignmentMissing`, causing a completeness failure rather than false acceptance.

Impact. A prover can produce a valid execution with a public return value different from the result expected by the verifier, while the verifier accepts without receiving that return value. An integration that treats successful verification as proof of a particular returned authorization decision can therefore accept the wrong result. This is a generic statement binding failure, although the reviewed credential circuits are not currently affected because they return no values.

The cardinality mismatch is primarily a validation weakness at the artifact boundary. The incomplete partition witness universe fails closed and affects completeness.

Recommendation. Derive the canonical public witness set directly from the union of `Circuit.public_parameters` and `Circuit.return_values`. Require the verifier to provide a value for every witness in that set, expose each one as a public R1CS input, and include the same ordered values in the proof transcript. Extend the ABI mapping to map `abi.return_type` onto `Circuit.return_values`. Reject artifacts unless every ABI parameter and return receives exactly its declared number of witnesses and no unexpected witnesses remain. Include all public parameters and return values in the partition allocation universe even when no opcode references them.

Client response. The client considers this finding outside the needs of the current application, since the reviewed credential circuits do not return values. No fix has been implemented.

LOW · #0B

Birth date parsing and age checks do not enforce calendar semantics

circuits/c0200_swiyu_jwt/src/main.nr,
circuits/c0202_sicpa_backend_constant/src/main.nr,
noir/c09_full_proof/src/main.nr

Description. The credential circuits do not fully validate birth date encodings, and the C09 circuit approximates calendar age using a fixed duration.

In C0200 and C0202, `birth_date_as_int` reads fixed positions from `dob_value.storage()` to construct a number in the form `YYYYMMDD`. It does not require the active value to contain exactly ten bytes, check the two separators or decimal digits, or validate the month, day, and leap year rules. The same parsing pattern appears in C0201 and C0203.

The disclosure hash commits only to the active contents of `dob_value`, while the age calculation reads its backing storage. Consequently, for a short disclosure, inactive bytes can influence the interpreted birth date without being included in the disclosure hash. This is distinct from the unchecked payload offsets: bounding payload reads alone does not validate the disclosure value.

In C09, the birth timestamp is parsed by subtracting ASCII `0` from each character without explicitly requiring decimal digits. The circuit then checks:

```
let seconds_per_year = 31557600;
let min_age_seconds = 18 * seconds_per_year;
assert(age_seconds >= min_age_seconds);
```

This duration is not always equivalent to the eighteenth calendar birthday. For example, a birth timestamp of January 1, 2000 at midnight UTC meets this threshold on December 31, 2017 at noon UTC, twelve hours before the eighteenth birthday.

C06 checks nonrevocation, not age. Its public date is used to check revocation list freshness, so it does not contain the birth date checks described here.

Impact. Assuming correct binding to the signed credential and disclosure, the parsing issues require an issuer to authenticate a malformed or short birth date value. They do not let a holder modify a correctly issued birth date. However, the circuit may accept an age statement based on a value that does not represent a valid birth date. Independently, C09 can accept a genuine credential before the holder's eighteenth calendar birthday, even when the verifier supplies the correct current time.

Recommendation. Require exactly ten active bytes for the `YYYY-MM-DD` disclosure, validate its digits and separators, and enforce valid Gregorian month and day ranges before calculating age. Validate C09's decimal timestamp encoding before conversion. Use calendar anniversaries instead of an average duration to enforce calendar age, with an explicit date convention and rule for February 29 birthdays. Require a valid verifier supplied current date and reject birth dates in the future.

Client response. The client is evaluating the most appropriate fixes for these issues.

LOW · #0C

Prepared proving state and debug logs expose private witness data

spartan-backend/src/precompute.rs, spartan-backend/src/circuit_instance.rs, spartan-backend/src/lib.rs

Description. The reusable `precompute.bin` artifact contains sensitive witness material. `PrecomputeFile` serializes the proving key, verification key, and Vega `PrepSnark` state. The prepared state includes invariant witness values, which for C0200 contain private credential payload data, disclosure data, salts, and issuer signature intermediates. Loading the artifact reconstructs a prover that can continue producing online proofs.

The artifact is written as ordinary, unencrypted bincode data without explicitly enforcing permissions that restrict access to its owner. Its confidentiality therefore depends on the surrounding filesystem permissions and how generated artifacts are handled.

Separately, the backend logs complete mapped prover inputs at debug level during circuit loading and proof generation. When debug logging is enabled, private witness values can consequently be copied into local logs or centralized logging systems.

Impact. Anyone able to read the prepared artifact or collected debug output may learn private credential and disclosure data. Treating the artifact as a public proving key or disposable cache can expose these values through shared build artifacts, backups, or log infrastructure with weaker access controls. This is an operational confidentiality issue; it does not by itself demonstrate a proof forgery or possession of the holder's device key.

Recommendation. Document `precompute.bin` as credential secret material and create it with restrictive permissions in a private directory. Exclude it from shared artifacts, caches, logs, and unrestricted backups. Where the deployment threat model requires encryption at rest, protect the

artifact with authenticated encryption and manage its key separately. Remove or redact all private witness values from diagnostic logs, including debug output.

Client response. The client plans to store the prepared proving state in private mobile application storage and is evaluating storage capacity and encryption requirements.

INFORMATIONAL · #0D

Standalone non revocation circuit limitations should be documented

noir/c06_non_revocation/src/main.nr

Description. `c06_non_revocation` isolates the non revocation check for benchmarking. It reads an identifier from the private `credential_string`, checks that the corresponding bit is clear in a signed revocation list, and verifies the list's signature and expiration condition. It intentionally does not authenticate the credential string or establish holder ownership.

The repository README describes separate proof components and a combined C09 circuit, but describes C06 as proving non revocation of the holder's credential. The WP6 documentation lists the checks performed without explicitly warning that the identifier is unauthenticated. C09 combines credential authentication, holder binding, age, and non revocation checks over the same credential witness.

Impact. This is an intended boundary of the benchmark circuit rather than an implementation defect. However, accepting C06 independently as evidence that a holder's issued credential is not revoked would establish a weaker statement: the prover can choose any identifier with a clear bit in the signed list.

Recommendation. Explicitly document C06 as an isolated benchmark component that authenticates the revocation list but not the credential or holder. State that production use requires credential authentication bound to the same identifier, as in C09. If separate proofs are used, they must be bound to the same credential; verifying them independently under the same issuer key is insufficient.

Client response. The client clarified that C06 is intended only to measure the cost of adding a non revocation check to the full proof, and that credential authentication is intentionally performed by the other components.

INFORMATIONAL · #0E

Partial base64url chunks use bytes beyond the logical payload

circuits/eid/src/fast_base64.nr

Description. Both reviewed circuits use `encode_url_into` to reconstruct the base64url-encoded payload before verifying the issuer's signature. The function processes the complete fixed-capacity input array and returns the number of characters corresponding to the logical length, `in_len`.

In the chunk containing the end of the logical payload, the function selects bytes according to the capacity `IN_LEN` rather than the logical length:

```
let b0 = if i0 < IN_LEN { input[i0] } else { 0 };
let b1 = if i0 + 1 < IN_LEN { input[i0 + 1] } else { 0 };
let b2 = if i0 + 2 < IN_LEN { input[i0 + 2] } else { 0 };
```

When the logical payload ends after the first byte of a three-byte chunk, the last returned base64url character depends on the upper four bits of `input[in_len]`. When it ends after the second byte, the last returned character depends on the upper two bits of `input[in_len]`. This byte is outside the logical payload and is controlled by the prover. The resulting characters can therefore contain nonzero base64url pad bits instead of being the canonical unpadded base64url encoding of `input[..in_len]`.

Impact. No way to bypass issuer authentication through this behavior was identified. The issuer's signature is still checked over the characters constructed by the circuit, so the prover must choose the extra bits to match the payload encoding that the issuer signed. For a canonical encoding, the affected bits must be zero. The current behavior nevertheless allows the signature check to depend on data outside the logical payload and permits non-canonical base64url encodings when an issuer signs one.

Recommendation. Ensure that the returned encoding depends only on bytes within `input[..in_len]`. The missing bytes of the final partial chunk can be treated as zero, or the unused bits of its final base64url character can be constrained to zero.

Client response. The client addressed this finding in PR #65 by making the base64url encoder use zero padding for bytes beyond the logical payload length.

KNOWN ISSUE · #0F

Curve operations do not enforce P256 point membership

spartan-backend/src/noir/synthesis/blackbox/ec_add.rs, spartan-backend/src/noir/synthesis/blackbox/multi_scalar_multiplication.rs

Description. The Spartan backend accepts arbitrary field coordinates as inputs to P256 addition and variable base scalar multiplication without proving that they represent points on P256.

The `EmbeddedCurveAdd` handler allocates the two coordinates of each input, sets the internal infinity flag to zero, and passes the resulting values to `AllocatedPoint::add`. The variable base MSM handler constructs its input point in the same way before calling `scalar_mul`. Neither path enforces the P256 curve equation

$$y^2 = x^3 - 3x + b.$$

The point gadgets enforce affine slope formulas that are valid for curve points, but those formulas can also be satisfied by arbitrary field coordinate pairs. For example, our proof of concept supplies the two points

```
P = (1, 1)
Q = (2, 2)
```

Neither point is on P256. Nevertheless, the backend applies the ordinary affine formula with slope one and accepts the result

```
x = p - 2
y = 2
```

through the normal ACIR to R1CS proving path. This demonstrates that the generated constraints prove arithmetic over arbitrary coordinate pairs rather than the specified P256 operation. We shared this proof of concept with the client.

The reviewed credential circuits pass security sensitive points into these handlers. The private `R_jwt` value enters the issuer signature equation in `c0200_swiyu_jwt` and `c0202_sicpa_backend_constant`. The holder equation also consumes point coordinates through MSM and curve addition. A malicious prover may therefore satisfy formulas that do not represent valid P256 signature relations.

This issue is separate from Point at infinity handling breaks valid curve operations. That finding concerns the correct handling of the valid group identity and causes valid operations to fail. This finding concerns nonidentity coordinate pairs that do not lie on the curve and can make invalid operations satisfy the backend constraints.

Impact. A malicious prover can produce a proof for an EC addition or variable base scalar multiplication result that is false under P256 semantics. Since the reviewed issuer and holder authentication checks rely on these operations, missing membership constraints weaken their signature equations and may allow authentication statements to be satisfied using invalid points. The proof of concept confirms false curve arithmetic through the production lowering, although the simplest confirmed issuer bypass uses the separate degenerate infinity witnesses described in Degenerate issuer signature witnesses bypass authentication.

Recommendation. Add a point validity gadget and invoke it for every untrusted input and result handled by EC addition and variable base MSM. For each finite point, constrain the P256 curve equation. Represent infinity explicitly, constrain its flag to be Boolean, and require its coordinates to use the canonical `(0,0)` encoding.

Client response. The client addressed this finding in PR #65 by adding P-256 membership constraints to EC addition and variable base MSM inputs, and validating constant points in the fixed base MSM path.

KNOWN ISSUE · #10

The demonstration verifier does not enforce policy or presentation freshness

preprocessing/c0200_siyu_jwt/src/main.rs, spartan-backend/src/circuit_instance.rs, spartan-backend/src/nizk_verifier.rs, noir/scripts/server/verifier-server.py

---

Description. Public circuit inputs are bound to a proof, but the circuit does not establish who selected them. A production verifier must compare every policy input with independently trusted values.

The C0200 demonstration instead uses prover side preprocessing to write the issuer key, current date, and the public device points `T_dev` and `U_dev` into `verifier_input.json`: `main.rs`. The backend then loads that file from the same circuit directory: `circuit_instance.rs`. Its verifier checks the proof and compares the public scalar vector with those loaded values, but it does not derive them from relying party policy or session state: `nizk_verifier.rs`.

This is particularly important for the holder check. The circuit proves only

$$x(s_{dev}T_{dev} + U_{dev}) = x(Q_{dev}).$$

It does not receive the verifier challenge or prove how `T_dev` and `U_dev` were constructed. Although a circuit comment assigns that validation to an external verifier, the demonstrated verification path does not implement it. If the prover may choose all three values, it can select a scalar and valid points that make the equation true without a device signature. For example, choosing `s_dev = 1`, `T_dev = G`, and `U_dev = Q_dev − G` satisfies the relation using valid P256 points. The same absence of trusted challenge state also permits replay whenever the verifier accepts the same public input vector.

The separate C06 and C09 demonstration service has the same trust boundary problem. It accepts the verification key and serialized public inputs with the request and forwards them to the proof verifier without pinning the intended circuit or comparing the issuer key, time, and challenge with server policy: `verifier-server.py`.

These behaviors are consistent with an early demonstration in which one local workflow performs both roles, but they are not a secure presentation protocol. The issue is not that the relevant values are public. It is that the demonstrated verifier does not establish that they came from the verifier or its trusted configuration.

Impact. If an integration reuses the demonstration flow as an authentication boundary, a prover can choose the issuer, asserted time, verification key, and holder proof values. In particular, it can satisfy the C0200 holder equation without possessing the device private key and can replay a presentation that is not bound to a fresh verifier session. A deployment that independently pins every policy value and validates the device points against a fresh challenge is not vulnerable to this integration failure.

Recommendation. Separate prover construction from verifier configuration. The verifier should pin the circuit verification key and accepted issuer keys, supply the current time, create a fresh challenge bound to the relying party, intended action, session, and expiration, and reject public inputs that do not match those values.

For the holder relation, either verify the complete device signature inside the circuit or have the verifier independently derive and validate `T_dev` and `U_dev` from the fresh challenge and signature data before accepting the proof. Bind the challenge or a collision resistant commitment to that session into the public statement and consume it after successful verification. The verifier API should require verifier controlled public inputs directly and should not load private prover witnesses or prover generated policy files.

Client response. The client acknowledges this limitation of the demonstration workflow. Verifier policy enforcement and presentation freshness remain to be addressed in the production integration.

KNOWN ISSUE · #11

Unchecked P256 points can be folded into false curve statements

noir/acvm-repo/t256_blackbox_solver/src/lib.rs

Description. The T256 Noir fork decodes P256 point coordinates with `Affine::new_unchecked`. The decoder does not check that finite input points lie on P256.

The same solver is used during compile time evaluation and MSM simplification. A fully constant EC addition or MSM is evaluated through this decoder and replaced by ordinary coordinate constants in ACIR. The generated circuit then contains neither the original operation nor its input points.

For example, the solver accepts the off curve input `(0,1)` for an enabled EC addition. The resulting coordinates are also treated as an ordinary computed result. When this occurs during constant folding, a proving backend cannot subsequently enforce point membership because the relevant curve operation has already been removed. We demonstrated this behavior with proof of concept programs shared with the client.

This is separate from Curve operations do not enforce P256 point membership. That finding concerns the Spartan lowering of retained ACIR curve operations. This finding concerns incorrect compiler and ACVM evaluation before an ACIR curve operation is emitted.

Impact. A circuit compiled by the T256 fork can enforce constants derived from an invalid P256 operation rather than the relation expressed in its Noir source. A backend may correctly prove the generated ACIR, but the proof does not establish the intended source level curve statement. The reviewed C0200 and C0202 circuits do not use this constant input pattern, so no direct impact on those circuits was demonstrated.

Recommendation. Use one strict T256 point decoder for runtime execution, compiler evaluation, and simplification. Reject finite input points unless their coordinates are canonical P256 field elements and they satisfy the P256 curve equation and subgroup requirements. Do not fold a curve operation whose inputs fail this validation.

Client response. The client addressed this finding in commit `ec7c9e9` by replacing `Affine::new_unchecked` with `Affine::new`, which checks curve and subgroup membership.